

To manage these specialized features within the **Lunar (Filament-based)** dashboard, you will leverage Filament's "Repeater" and "Color Picker" fields. This allows you to build a powerful "Store Settings" page where an admin can configure a school or business store without touching code.

Here is the technical breakdown for the Filament Resource and the corresponding Blade implementation.

1. The Lunar Dashboard (Filament Resource)

You will create a StoreResource.php. This is where you manage the Multi-Tenancy (MT) settings.

PHP

```
// app/Filament/Resources/StoreResource.php
public static function form(Form $form): Form
{
    return $form->schema([
        Forms\Components\Tabs::make('Store Configuration')->tabs([
            // Tab 1: Visual Identity
            Forms\Components\Tabs\Tab::make('Branding')->schema([
                Forms\Components\FileUpload::make('logo_path')->directory('logos'),
                Forms\Components\Select::make('nav_layout')
                    ->options(['left' => 'Logo Left', 'center' => 'Centered', 'right' => 'Logo Right']),
                Forms\Components\ColorPicker::make('colors.primary')->label('Primary Color'),
                Forms\Components\ColorPicker::make('colors.secondary')->label('Secondary Color'),
                Forms\Components\ColorPicker::make('colors.accent')->label('Accent Color'),
                Forms\Components\Select::make('font_family')
                    ->options(['Inter' => 'Modern', 'Roboto' => 'Clean', 'Merriweather' => 'Classic']),
            ]),

            // Tab 2: Events & Calendar
            Forms\Components\Tabs\Tab::make('Events')->schema([
                Forms\Components\Repeater::make('events')
                    ->schema([
                        Forms\Components\TextInput::make('name')->required(),
                        Forms\Components\DateTimePicker::make('event_date')->required(),
                        Forms\Components\Toggle::make('show_countdown')->label('Show Countdown?'),
                    ])
                    ->columns(3),
            ]),
        ]),
    );
}
```

```

    ]),
    // Tab 3: Roster
    Forms\Components\Tabs\Tab::make('Roster')->schema([
        Forms\Components\Repeater::make('roster')
            ->schema([
                Forms\Components\FileUpload::make('photo')->image(),
                Forms\Components\TextInput::make('name'),
                Forms\Components\TextInput::make('meta')->placeholder('Grade or Department'),
            ])>grid(2),
    ]),
    ],
    ],
    ];
}

```

2. The Multi-Tenant Blade Structure

You don't want to create separate Blade files for 100 stores. Instead, create **flexible components** that adapt based on the Store model.

Calendar & Countdown Component

Using Alpine.js for the countdown ensures it is reactive and doesn't require a page refresh.

HTML

```

{{-- resources/views/components/event-countdown.blade.php --}}
@php $nextEvent = collect(app('current_store')->events)->firstWhere('show_countdown', true);
@endphp

@if($nextEvent)
<div x-data="countdown('{{ $nextEvent['event_date'] }}'" class="p-6 bg-brand-accent text-white text-center">
    <h3 class="text-xl font-bold">Next Event: {{ $nextEvent['name'] }}</h3>
    <div class="flex justify-center gap-4 text-2xl">
        <div><span x-text="days"></span>d</div>
        <div><span x-text="hours"></span>h</div>
        <div><span x-text="minutes"></span>m</div>
    </div>

```

```
</div>
```

```
<script>
```

```
function countdown(date) {
```

```
  return {
```

```
    days: 0, hours: 0, minutes: 0,
```

```
    init() {
```

```
      setInterval(() => {
```

```
        let diff = new Date(date) - new Date();
```

```
        this.days = Math.floor(diff / (1000 * 60 * 60 * 24));
```

```
        this.hours = Math.floor((diff % (1000 * 60 * 60 * 24)) / (1000 * 60 * 60));
```

```
        this.minutes = Math.floor((diff % (1000 * 60 * 60)) / (1000 * 60));
```

```
      }, 1000);
```

```
    }
```

```
  }
```

```
}
```

```
</script>
```

```
@endif
```

3. Specialized Roster Gallery

For school stores, displaying the roster (students or faculty) creates a personalized feel.

HTML

```
{{-- resources/views/components/roster-grid.blade.php --}}
```

```
<section class="grid grid-cols-2 md:grid-cols-4 gap-6 p-8">
```

```
  @foreach(app('current_store')->roster as $person)
```

```
    <div class="text-center">
```

```
      
```

```
      <h4 class="mt-2 font-bold">{{ $person['name'] }}</h4>
```

```
      <p class="text-sm text-gray-500">{{ $person['meta'] }}</p>
```

```
    </div>
```

```
  @endforeach
```

```
</section>
```

4. Lunar Catalog Scoping

When a user visits school-a.top5pct.com, you must ensure they only see products for "School A."

In your Livewire component that displays the catalog, you will hook into Lunar's Product query:

PHP

```
// app/Http/Livewire/ProductIndex.php
public function getProductsProperty()
{
    return \Lunar\Models\Product::active()
        ->whereHas('channels', function ($query) {
            $query->where('handle', app('current_store')->channel->handle);
        })
        ->paginate(12);
}
```

Summary of Area Focus

1. **Shared Layout:** Use one app.blade.php that loads CSS variables dynamically from your Store model.
2. **Lunar Customization:** Extend Lunar by adding a Store relationship to your Products. This allows you to say "This Nike Hoodie belongs to both the School Store and the General Store."
3. **Deployment (Forge/Vapor):** Since you are using subdomains, you need a **Wildcard SSL**.
 - In **Forge**, you can enable "Wildcard Subdomains" which will automatically provision an SSL certificate for *.top5pct.com.
 - Any request to anything.top5pct.com will hit your app, and your Middleware will do the lookup.

Next Step Recommendation: Start by building the Store migration and the Middleware.

Without the "Context Switch" working, the Blade components won't have any data to pull from. Do you have your stores table migration ready, or should we draft that?